

Guia de Configuração das Chaves de API — Central da Nação (.env / GitHub)

Documento de Integração e Finalização · pt-BR · 2026-09-09

Objetivo: listar TODAS as variáveis do MVP, onde obtê-las, onde configurá-las e quais são opcionais. Nenhum segredo real aparece neste documento.

Regra-mãe: secrets só em `.env` (máquina local) ou GitHub Secrets — **nunca** no git.

1. Onde cada chave vive

Onde	O que vai	Uso
<code>.env</code> na pasta <code>tools/</code> (máquina local/editor)	<code>SUPABASE_URL</code> , <code>SUPABASE_KEY</code> , <code>API_FUTEBOL_KEY</code> , <code>LLM_API_KEY</code> , <code>EDIT-OR_WEBHOOK_URL</code> , <code>AUTO_PUBLISH</code> , <code>DEFAULT_CITY</code>	tools Python (ingestão/classificação/publish)
GitHub → Settings → Secrets and variables → Actions → Secrets	mesmos nomes acima + <code>NEXT_PUBLIC_SUPABASE_URL</code> , <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	workflows do GitHub Actions (crons e build da Web App)
GitHub → Variables	<code>AUTO_PUBLISH</code> , <code>DEFAULT_CITY</code>	valores não-secrets lidos pelos workflows
Supabase → Project Settings → API	URL e chaves	base do banco (service role p/ tools; anon p/ Web App)

2. Variáveis do pipeline (contrato `tools/.env.example`)

`SUPABASE_URL` — obrigatória para cloud

- Onde: Supabase Dashboard → **Project Settings** → **API** → Project URL (ex.: `https://xxxx.supabase.co`).
- Passo: criar o projeto no [supabase.com](#) e aplicar os scripts de `supabase/01_schema.sql` e `supabase/02_seed_allowlists.sql` no **SQL Editor** antes de rodar o pipeline.

`SUPABASE_KEY` — obrigatória para cloud (tools)

- Onde: mesmo painel → `service_role` **secret** (chave de serviço).
- ⚠ A service key ignora RLS: use-a SOMENTE em código server-side/tools/GitHub Secrets. **Nunca** exponha no Web App (client) — lá vai a anon key.

API_FUTEBOL_KEY — obrigatória no MVP (decidir provedor)

- Provedores possíveis (escolher UM — decisão humana registrada em `findings.md` §5.1):
 1. **api-futebol.com.br** (API brasileira) — key em <https://api-futebol.com.br> (painel do assinante).
 2. **api-sports.io (API-Football)** — key em <https://www.api-football.com> (Dashboard → Account → keys).
- Sem a key, o pipeline roda em **mock com fixtures** (nada de mock é sincronizado na nuvem — guard nos workflows).
- Variável: `API_FUTEBOL_KEY=<sua chave>`.

LLM_API_KEY — obrigatória no MVP (escolher provedor)

- OpenAI (<https://platform.openai.com/api-keys>), Anthropic (<https://console.anthropic.com/settings/keys>) ou Google Gemini (<https://aistudio.google.com/apikey>).
- A tool usa a variável genérica `LLM_API_KEY` (ver `tools/llm_adapter.py` para o formato esperado).
- Sem a key, a classificação usa o modo mock e marca `needs_review: true`.

EDITOR_WEBHOOK_URL — obrigatória para a fila editorial

- Slack: criar **Incoming Webhook** em <https://api.slack.com/apps> → seu app → Incoming Webhooks → copiar URL.
- Discord: Server Settings → **Integrations** → **Webhooks** → **New Webhook** → copiar URL.
- A tool posta `{"content": "<bloco Hoje na Nação>"}` — formato compatível com os dois.
- Sem URL, os workflows rodam `--dry-run` e a fila fica registrada no artefato (nenhum bloco é enviado).

AUTO_PUBLISH — variável de controle (default `false`)

- `false` (MVP): payload só vai para a home com aprovação explícita do editor.
- `true`: o publish das 07:45 publica payload válido sem esperar reação.
- Configurar como **Variable** no GitHub (não Secret): `AUTO_PUBLISH=false`.

DEFAULT_CITY — cidade padrão do módulo Perto de você (default Rio de Janeiro)

- Valores conhecidos (cidades piloto): `Rio de Janeiro`, `Belo Horizonte`, `Recife`, `Salvador`, `São Paulo`.

3. Variáveis da Web App (Next.js — GitHub Pages)

Definidas como **Secrets** do repositório (são injetadas em tempo de build pelos workflows):

Secret	Valor	Observação
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Project URL do Supabase	mesma de <code>SUPABASE_URL</code>
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	chave anon/public (<code>anon public</code>) do painel API	segura para o client; RLS limita o que ela lê (só publicado/aprovado)

Se a Web App usar leitura server-side (revalidação/ISR não se aplica em export estático), ajustar os nomes no workflow `central-da-nacao-deploy-webapp.yml` conforme o que o time da Web App definir.

4. Configuração no GitHub (passo a passo)

1. **Criar o repositório** (monorepo) e enviar: `.github/`, `supabase/`, `tools/`, `fixtures/`, `webapp/`, `architecture/`, `docs`.
2. **Secrets** — repo → Settings → Secrets and variables → Actions → New repository secret (um por vez):
`SUPABASE_URL`, `SUPABASE_KEY`, `API_FUTEBOL_KEY`, `LLM_API_KEY`, `EDITOR_WEBHOOK_URL`,
`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`.
3. **Variables** — mesma tela, aba Variables: `AUTO_PUBLISH=false`, `DEFAULT_CITY=Rio de Janeiro`.
4. **GitHub Pages** — repo → Settings → Pages → **Source: GitHub Actions**.
5. **Conferir Actions** — aba Actions: os workflows `central-da-nacao-diario`, `central-da-nacao-matchday` e `central-da-nacao-deploy-webapp` devem aparecer. Rodar manualmente `central-da-nacao-diario` com job `ingest` (teste) e job `publish` com `decision=approved` (teste do gate).
6. **Aprovação manual do editor (MVP)** — Actions → `central-da-nacao-diario` → **Run workflow** → job: `publish` → decision: `approved`. Alternativa futura: botões  no Slack/Discord via callback (fora do escopo MVP).

5. Fase 2 (NÃO configurar agora)

`RESEND_API_KEY` (digest e-mail), `YOUTUBE_API_KEY`, `GOOGLE_MAPS_KEY` (geocoding), WhatsApp Cloud API — ver tabela de Integrations do blueprint.

6. Checklist de aplicação deste guia

- [] `supabase/01_schema.sql` e `02_seed_allowlists.sql` executados no projeto Supabase
- [] `tools/.env` criado a partir de `tools/.env.example` com `SUPABASE_URL` + `SUPABASE_KEY`
- [] `python3 tools/ping_supabase.py` → write/read OK em `agent_pings`
- [] Segredos e variáveis criados no GitHub (seção 4)
- [] Primeiro deploy do GitHub Pages OK; home abre com o último payload `published` (ou estado vazio inicial)
- [] `AUTO_PUBLISH` permanece `false` até decisão do editor (blueprint)